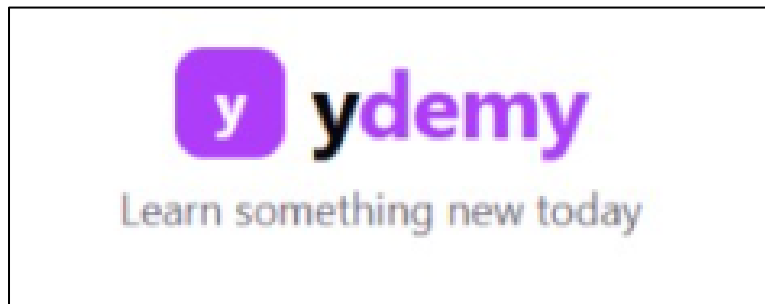




IPT CAPSTONE PROJECT

E-Learning Application with AWS CloudFormation, Jenkins CI/CD, Docker, S3 and Monitoring



Done by

Santhoshi Ravi

Neesha

Karthick

TABLE OF CONTENTS

Chapter No	Title
1	Introduction
2	System Architecture
3	Application Design and Implementation
4	AWS Cloud Infrastructure using Cloud Formation
5	Dockerization and CI/CD Pipeline
6	Monitoring and Observability
7	Conclusion and Future Scope
8	Appendix

CHAPTER 1

INTRODUCTION

1.1 Project Overview

This project implements an end-to-end DevOps solution for an E-Learning Platform. The application enables students to register, log in, view available courses, enroll in courses and access course content. Administrators can log in, create courses and upload learning materials. The application is intentionally simple from a business-feature perspective so that the main focus remains on cloud infrastructure, automated deployment, secure storage and monitoring.

The completed system uses React for the frontend, Spring Boot for backend APIs, PostgreSQL for relational data, Amazon S3 for course content storage, Docker for containerization, Jenkins for CI/CD, Docker Hub for image registry, AWS CloudFormation for infrastructure provisioning, and Prometheus and Grafana for monitoring.

1.2 Problem Statement

Traditional manual deployment is time-consuming, error-prone and difficult to repeat consistently. A working application also needs secure storage, controlled access, infrastructure automation and operational visibility. This project addresses these concerns by implementing a complete DevOps lifecycle around an E-Learning Platform.

1.3 Objectives

- Develop a working E-Learning Platform with Student and Admin roles.
- Implement JWT-based authentication and role-based authorization.
- Store structured application data in PostgreSQL.
- Store uploaded course PDFs and videos in Amazon S3.
- Use pre-signed URLs for secure temporary access to course content.
- Containerize frontend and backend using Docker.
- Provision AWS infrastructure using CloudFormation templates.
- Automate build, image push and deployment using Jenkins.
- Deploy the system across Frontend, Backend, Jenkins and Monitoring EC2 instances.
- Monitor application and infrastructure metrics using Prometheus and Grafana.

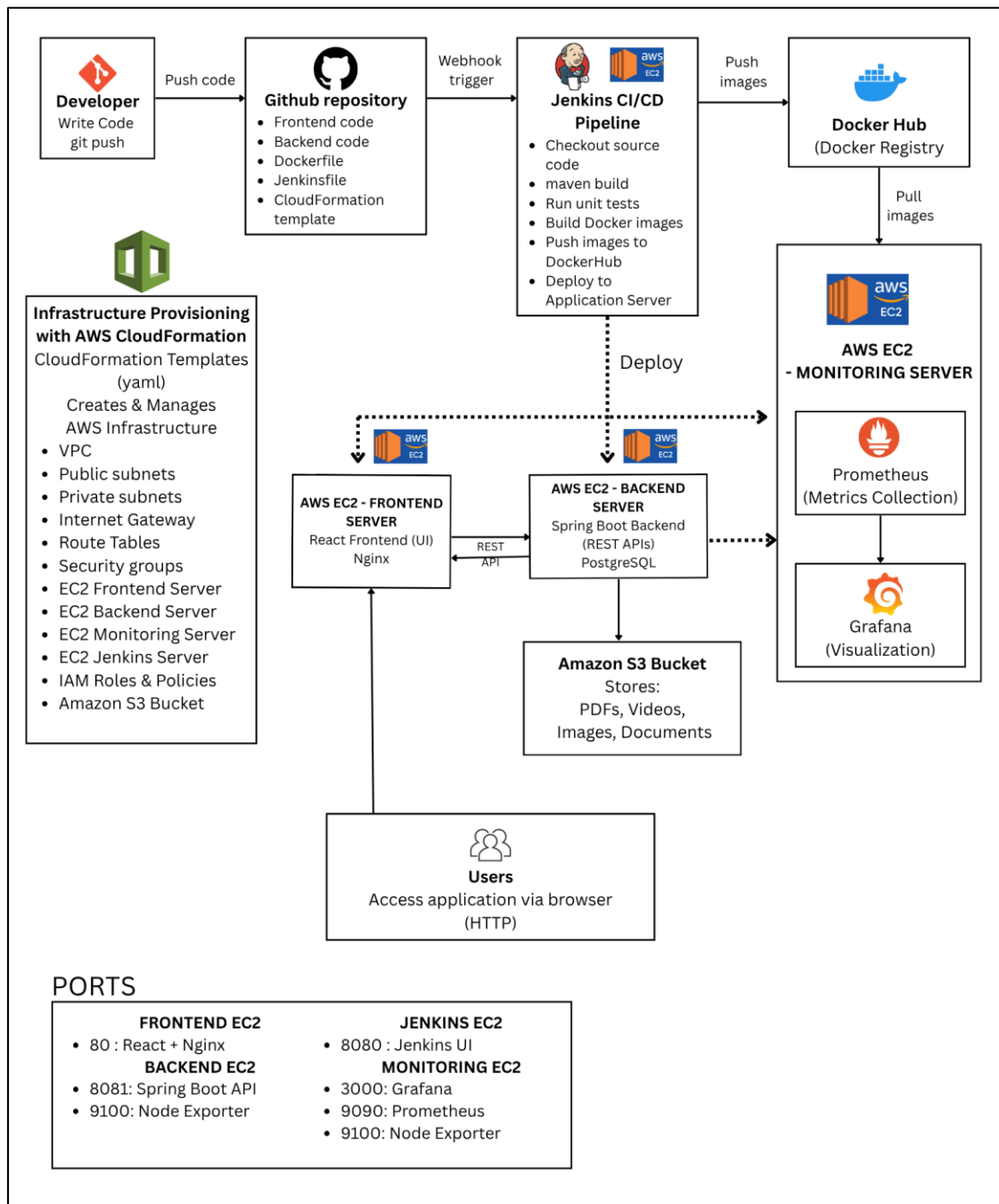
1.4 Technology Stack

Layer	Technologies
Frontend	React, JavaScript, React Router, Axios, Nginx
Backend	Java 21, Spring Boot, Spring Security, JWT, JPA, Hibernate, Flyway, Maven
Database	PostgreSQL 16
Storage	Amazon S3, pre-signed URLs
Infrastructure	AWS EC2, VPC, Subnet, Internet Gateway, Security Groups, IAM, Elastic IP, CloudFormation
Containerization	Docker, Docker Compose
CI/CD	GitHub, Jenkins, Docker Hub, SSH deployment
Monitoring	Prometheus, Grafana, Node Exporter, Spring Boot Actuator, Micrometer

CHAPTER 2

SYSTEM ARCHITECTURE

2.1 Architecture Diagram



The final architecture uses four AWS EC2 instances and one Amazon S3 bucket. The Frontend EC2 hosts the React application through an Nginx container. The Backend EC2 hosts the Spring Boot API, PostgreSQL database container and Node Exporter. The Jenkins EC2 runs the CI/CD pipeline, builds application artifacts, creates Docker images, pushes them to Docker Hub and deploys containers to the target EC2 instances over SSH. The Monitoring EC2 runs Prometheus and Grafana and scrapes metrics from all exporter targets.

Component	Final Responsibility
Frontend EC2	Runs the frontend Docker container. The React production build is served by Nginx on port 80. Node Exporter exposes host metrics on port 9100.
Backend EC2	Runs Spring Boot backend on port 8081, PostgreSQL database container, Amazon S3 integration and Node Exporter.
Jenkins EC2	Runs Jenkins, Java 21, Maven, Node.js, Docker, Docker Compose and performs build, push and deployment automation.
Monitoring EC2	Runs Prometheus, Grafana and Node Exporter. Prometheus scrapes backend application metrics and EC2 host metrics.
Amazon S3	Stores course content files such as PDFs and videos. Files are stored privately and accessed using pre-signed URLs.
Docker Hub	Stores frontend and backend Docker images tagged with Jenkins build number and latest.

2.2 Network and Host Layout

The AWS network uses a VPC with CIDR block 10.0.0.0/16 and a public subnet with CIDR block 10.0.1.0/24. The private IPs used in Jenkins deployment and Prometheus configuration are within this subnet, including Frontend EC2 at 10.0.1.69, Backend EC2 at 10.0.1.130 and Monitoring EC2 at 10.0.1.216.

Host	Private IP	Primary Ports
Frontend EC2	10.0.1.69	80 for frontend, 9100 for Node Exporter
Backend EC2	10.0.1.130	8081 for Spring Boot, 5432 for PostgreSQL container, 9100 for Node Exporter
Monitoring EC2	10.0.1.216	9090 for Prometheus, 3000 for Grafana, 9100 for Node Exporter
Jenkins EC2	CloudFormation output	8080 for Jenkins UI and GitHub webhook

2.3 Application Request Flow

1. User accesses the frontend through the Frontend EC2 public endpoint on port 80.
2. React frontend sends API requests to the Spring Boot backend.
3. Spring Security validates the JWT token for protected APIs.
4. Backend reads or writes application data in PostgreSQL.
5. Admin-uploaded content is stored in the private Amazon S3 bucket.
6. When content is requested, the backend checks enrollment and generates a pre-signed S3 URL.
7. The frontend opens the pre-signed URL and the browser retrieves the file directly from S3.

2.4 CI/CD Flow

1. Developer pushes code to GitHub.
2. GitHub webhook triggers the Jenkins pipeline.
3. Jenkins checks out source code and builds backend and frontend.
4. Jenkins builds Docker images for backend and frontend.
5. Images are pushed to Docker Hub with build number and latest tags.
6. Jenkins copies Docker Compose files to the EC2 servers using SCP.
7. Jenkins connects over SSH and restarts services using Docker Compose.
8. Jenkins validates backend, Prometheus and Grafana health endpoints

CHAPTER 3

APPLICATION DESIGN AND IMPLEMENTATION

3.1 User Roles and Features

Role	Features
Student	Signup, login, view available courses, enroll in courses, view enrolled courses and access course content.
Admin	Login, create courses, upload PDF/video course content and manage course material.

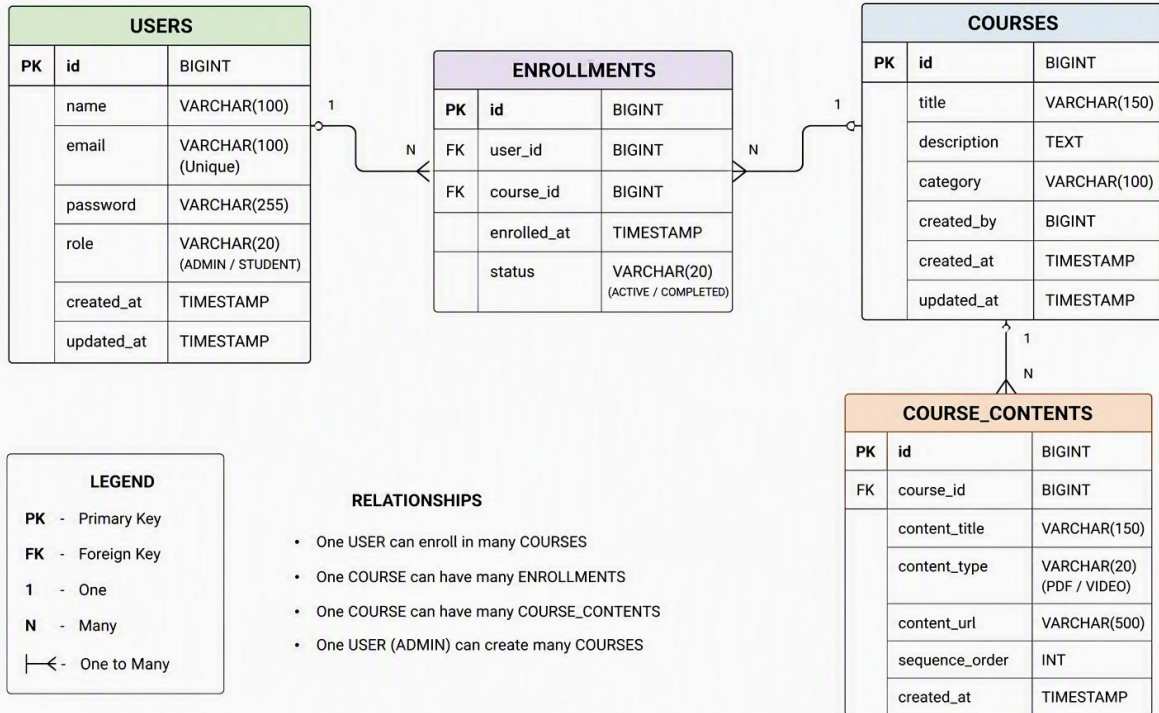
3.2 Backend Modules

Module / Package	Purpose
auth	Signup, login, authentication requests and responses.
security	JWT generation, JWT filter, Spring Security configuration and user details service.
user	User entity, role enum and user repository.
course	Course entity, course content entity, course APIs and course service logic.
enrollment	Enrollment entity, enrollment APIs and duplicate enrollment validation.
storage	FileStorageService abstraction, S3 storage implementation and pre-signed URL generation.
common	Common API response and centralized exception handling.
config	Admin bootstrap and application configuration classes.

3.3 Database Design

Table	Purpose	Important Fields
users	Stores student and admin accounts.	id, name, email, password_hash, role, created_at, updated_at
courses	Stores course metadata.	id, title, description, category, created_by, is_active
course_contents	Stores S3 object key and content metadata.	id, course_id, content_title, content_type, content_url, sequence_order
enrollments	Stores student-course relationship.	id, user_id, course_id, enrolled_at, status

E-LEARNING PLATFORM – ER DIAGRAM



3.4 API Summary

Category	Method	Endpoint	Purpose
Authentication	POST	/api/auth/signup	Register a user.
Authentication	POST	/api/auth/login	Authenticate user and return JWT.
Course	GET	/api/courses	View available courses.
Course	GET	/api/courses/{id}	View course details.
Enrollment	POST	/api/enrollments	Enroll in a course.
Admin Course	POST	/api/admin/courses	Create a new course.
Admin Course	PUT	/api/admin/courses/{id}	Update course details.
Admin Course	DELETE	/api/admin/courses/{id}	Delete or deactivate a course.
Course Content	POST	/api/admin/course-contents	Upload course content.
Course Content	GET	/api/course-contents/{courseId}	View content for a course.
Monitoring	GET	/actuator/health	Backend health check.
Monitoring	GET	/actuator/prometheus	Prometheus metrics endpoint.

3.5 S3 Content Access Flow

1. Admin uploads a PDF or video through the application.
2. Spring Boot validates the file and uploads it to Amazon S3.
3. The S3 object key is stored in PostgreSQL, not a public URL.
4. When a student requests content, the backend verifies enrollment.
5. The backend generates a pre-signed S3 URL valid for 10 minutes.
6. The frontend opens the pre-signed URL to access the file securely.

3.6 Frontend Design

Page / Component	Purpose
Login Page	Authenticates student or admin users.
Signup Page	Registers student users.
Courses Page	Displays available courses and supports enrollment.
My Courses Page	Displays enrolled courses.
Course Content Page	Displays content links for enrolled course.
Admin Dashboard	Creates courses and uploads course content.
Navbar	Provides navigation and role-aware admin access.

3.7 Security Design

Area	Security Measure
Authentication	JWT-based stateless authentication using Spring Security.
Authorization	Role-based access for Student and Admin features.
S3 Storage	Private bucket, blocked public access, encrypted objects and pre-signed URLs.
IAM	Backend EC2 role receives S3 permissions through IAM policy, avoiding hardcoded AWS keys.
Network	Security groups restrict access between frontend, backend, Jenkins and monitoring components.
Monitoring Access	Prometheus and Grafana access restricted using administrator IP.
Deployment Secrets	Docker Hub token and SSH key stored in Jenkins credentials.

CHAPTER 4

AWS CLOUD INFRASTRUCTURE USING CLOUD FORMATION

4.1 CloudFormation Template Set

The infrastructure is provisioned using five CloudFormation templates. The templates separate network, security, compute, Jenkins and storage concerns. This modular design keeps each template focused and easier to understand.

Template	Purpose
01-network.yaml	Creates VPC, public subnet, Internet Gateway, route table and route association.
02-security-groups.yaml	Creates security groups for frontend, backend, Jenkins, monitoring and SSH access.
03-ec2.yaml	Creates frontend, backend and monitoring EC2 instances, IAM role, instance profile and Elastic IPs.
04-jenkins.yaml	Creates Jenkins EC2 server, Jenkins role, instance profile, Elastic IP and bootstrap software.
05-storage.yaml	Creates S3 bucket, bucket policy and S3 access policy for backend EC2 role.

4.2 Network Stack

Resource	Configuration
VPC	CIDR 10.0.0.0/16 with DNS support and DNS hostnames enabled.
Public Subnet	CIDR 10.0.1.0/24 with public IP assignment on launch.
Internet Gateway	Attached to the VPC for internet connectivity.
Route Table	Default route 0.0.0.0/0 through Internet Gateway.

4.3 Security Group Design

Security Group	Main Rules
Frontend SG	Allows HTTP 80 and HTTPS 443 from internet; Node Exporter 9100 from Monitoring SG.
Backend SG	Allows Spring Boot 8081 from Frontend SG, Monitoring SG and Jenkins SG; Node Exporter 9100 from Monitoring SG.
Jenkins SG	Allows Jenkins UI/webhook on 8080 according to configured rules.
Monitoring SG	Allows Grafana 3000 and Prometheus 9090 from administrator IP.
SSH SG	Allows SSH 22 from administrator IP.

4.4 EC2 Stack

EC2 Instance	Provisioned Software and Purpose
Frontend EC2	Amazon Linux 2023, Docker, Git, AWS CLI, Docker Compose plugin. Hosts frontend container and Node Exporter.
Backend EC2	Amazon Linux 2023, Docker, Git, AWS CLI, Java 21, Docker Compose plugin. Hosts backend, PostgreSQL and Node Exporter.
Monitoring EC2	Amazon Linux 2023, Docker, Git, AWS CLI, Java 21, Docker Compose plugin. Hosts Prometheus, Grafana and Node Exporter.

4.5 Jenkins Stack

The Jenkins stack provisions a dedicated Jenkins EC2 instance with Java 21, Maven, Node.js 20, Docker, Docker Compose, Git, AWS CLI and Jenkins. A 30 GB gp3 root volume is configured, and a 2 GB swap file is created to reduce memory pressure during Maven, npm and Docker build stages on t3.micro.

4.6 S3 Storage Stack

S3 Capability	Implementation
Private Bucket	Public ACLs and public policies are blocked.
Encryption	Server-side encryption using AES256.
Versioning	Enabled to preserve object versions.
Lifecycle	Deletes non-current versions after 30 days and aborts incomplete multipart uploads after 7 days.
HTTPS Enforcement	Bucket policy denies requests where aws:SecureTransport is false.
IAM Access	Policy attached to elearning-ec2-role permits ListBucket, GetObject, PutObject and DeleteObject.

CHAPTER 5

DOCKERIZATION AND CI/CD PIPELINE

5.1 Docker Images

Image	Purpose
santhr/elearning-backend:latest	Spring Boot backend application container.
santhr/elearning-frontend:latest	React production build served by Nginx.
postgres:16	PostgreSQL database container on Backend EC2.
prom/node-exporter:latest	Host metrics exporter on frontend, backend and monitoring EC2 instances.
prom/prometheus:latest	Metrics collection on Monitoring EC2.
grafana/grafana:latest	Metrics visualization on Monitoring EC2.

5.2 Backend Docker Compose Deployment

The Backend EC2 runs PostgreSQL, the Spring Boot backend and Node Exporter. The backend container uses S3 storage mode and connects to PostgreSQL through the Docker network using the service name postgres.

Service	Port / Configuration
postgres	5432:5432, persistent volume postgres_data, health check using pg_isready.
backend	Image santhr/elearning-backend:latest, exposed on 8081:8081, STORAGE_TYPE=s3, S3 bucket configured.
node-exporter	9100:9100, host pid namespace and host filesystem mounts for EC2 metrics.

5.3 Frontend Docker Compose Deployment

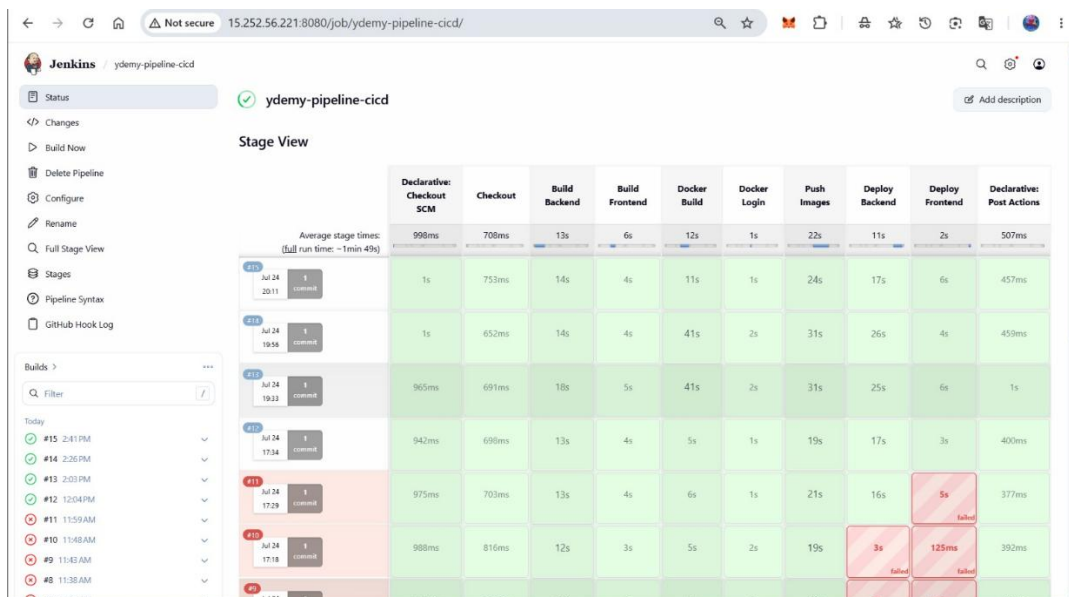
Service	Port / Configuration
frontend	Image santhr/elearning-frontend:latest, exposed on 80:80.
node-exporter	Exposes frontend EC2 metrics on 9100.

5.4 Monitoring Docker Compose Deployment

Service	Port / Configuration
prometheus	9090:9090, configuration mounted from prometheus.yml, persistent prometheus_data volume.
grafana	3000:3000, persistent grafana_data volume, demo admin credentials configured.
node-exporter	Exposes monitoring EC2 metrics on 9100.

5.5 Jenkins Pipeline Stages

Stage	Logic
Checkout	Retrieves source code from GitHub.
Build Backend	Runs Maven package command in backend directory using Java 21.
Build Frontend	Runs npm install and npm run build in frontend directory.
Docker Build	Builds backend and frontend images with BUILD_NUMBER and latest tags.
Docker Login	Authenticates with Docker Hub using Jenkins credentials.
Push Images	Pushes both versioned and latest image tags to Docker Hub.
Deploy Backend	Copies backend Compose file to Backend EC2 and runs docker compose pull and up.
Deploy Frontend	Copies frontend Compose file to Frontend EC2 and restarts frontend service.
Deploy Monitoring	Copies monitoring Compose and Prometheus config to Monitoring EC2 and restarts monitoring stack.
Health Check	Checks backend actuator health, Prometheus health and Grafana API health.



CHAPTER 6

MONITORING AND OBSERVABILITY

6.1 Monitoring Stack

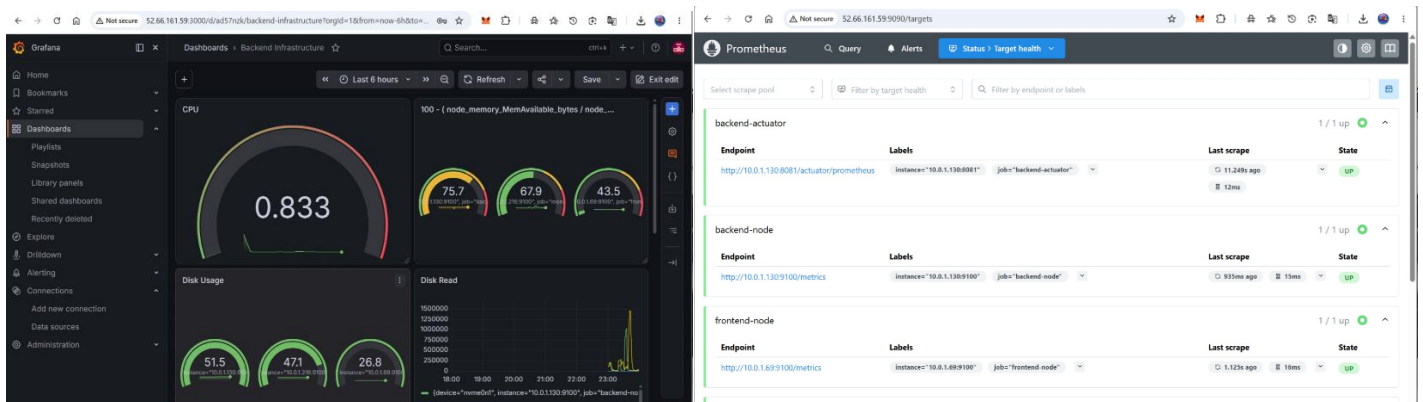
Monitoring is centralized on the Monitoring EC2 instance. Prometheus scrapes metrics from itself, the Monitoring EC2 Node Exporter, the Frontend EC2 Node Exporter, the Backend EC2 Node Exporter and the Spring Boot Actuator Prometheus endpoint. Grafana reads from Prometheus and displays dashboards.

6.2 Prometheus Scrape Targets

Job Name	Target	Purpose
prometheus	localhost:9090	Prometheus self-monitoring.
monitoring-node	10.0.1.216:9100	Monitoring EC2 host metrics.
frontend-node	10.0.1.69:9100	Frontend EC2 host metrics.
backend-node	10.0.1.130:9100	Backend EC2 host metrics.
backend-actuator	10.0.1.130:8081/actuator/prometheus	Spring Boot JVM, HTTP and application metrics.

6.3 Metrics Collected

Metric Category	Examples
Application Metrics	JVM memory, thread count, garbage collection, HTTP request count, response latency, application health.
Infrastructure Metrics	CPU utilization, memory usage, disk usage, filesystem usage, network traffic, system load.
Operational Health	Backend actuator health, Prometheus health endpoint and Grafana health endpoint.



CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project successfully implements a working E-Learning Platform supported by a complete DevOps workflow. The system includes application development, cloud infrastructure provisioning, Docker-based deployment, automated CI/CD, secure S3 storage and centralized monitoring. It demonstrates how a full-stack application can be built and operated using production-oriented DevOps practices.

7.2 Outcomes

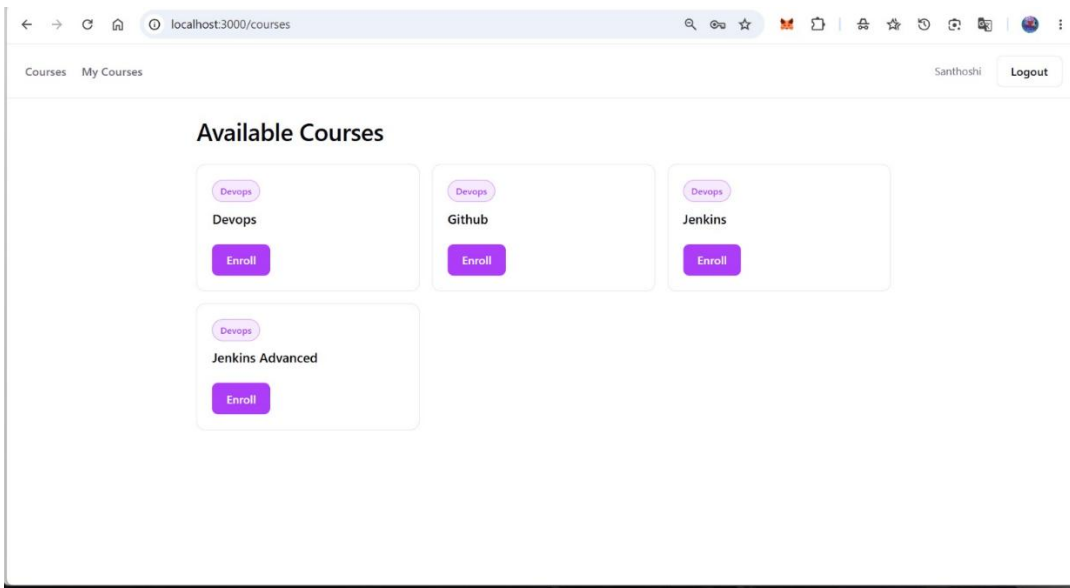
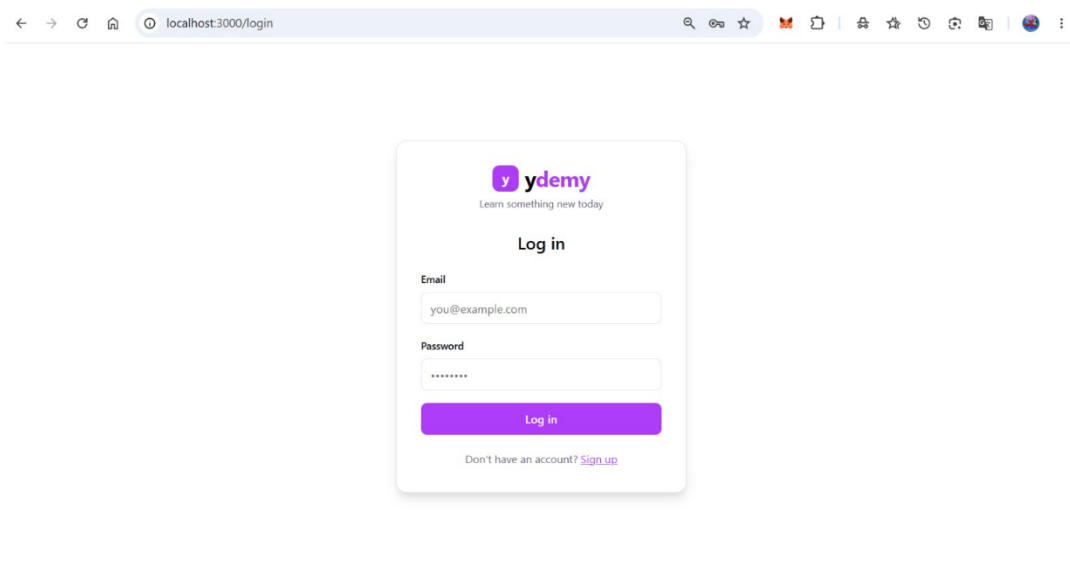
- Implemented frontend, backend, database and storage integration.
- Provisioned AWS infrastructure using CloudFormation templates.
- Automated Docker image build and deployment using Jenkins.
- Used Docker Hub as container registry.
- Deployed services across four EC2 instances.
- Integrated S3 storage with pre-signed URLs.
- Configured Prometheus and Grafana monitoring.

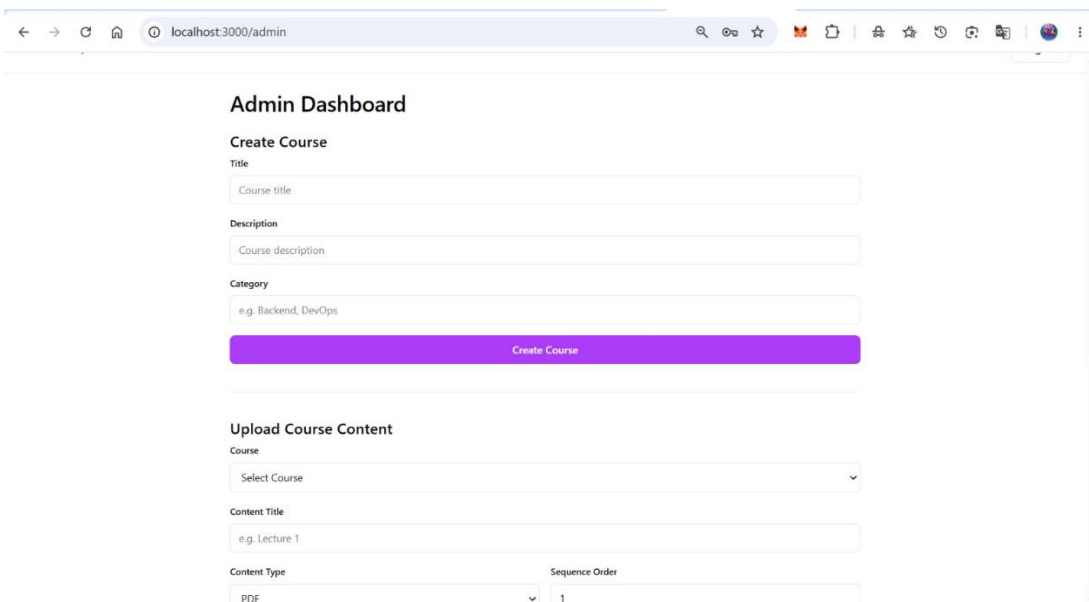
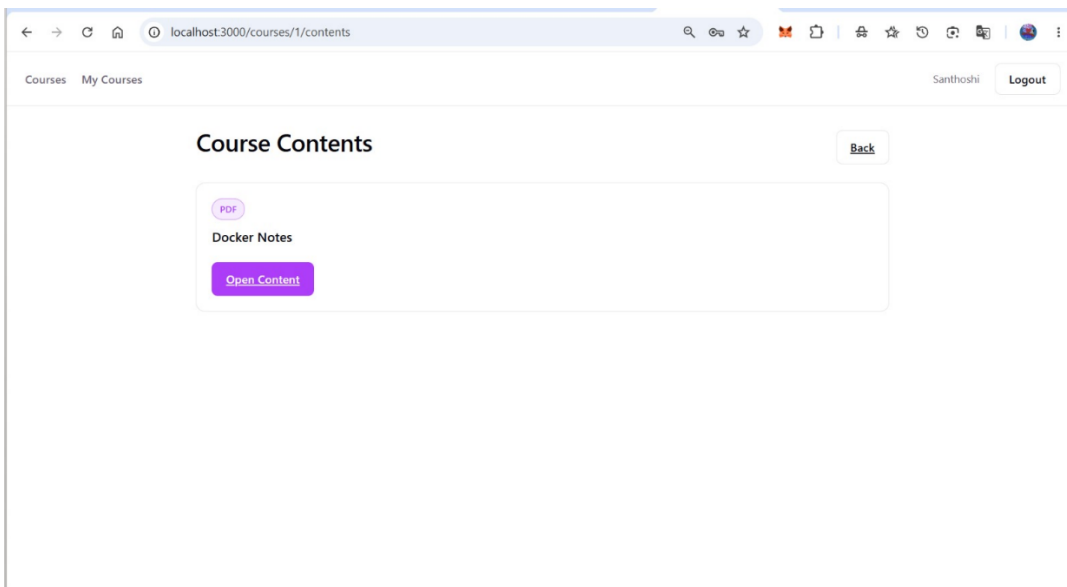
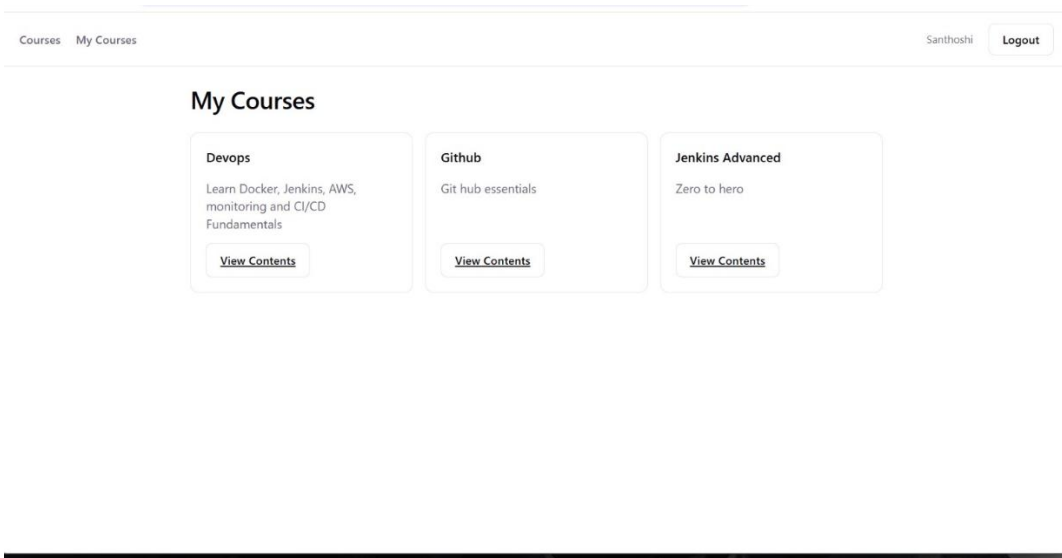
7.3 Future Scope

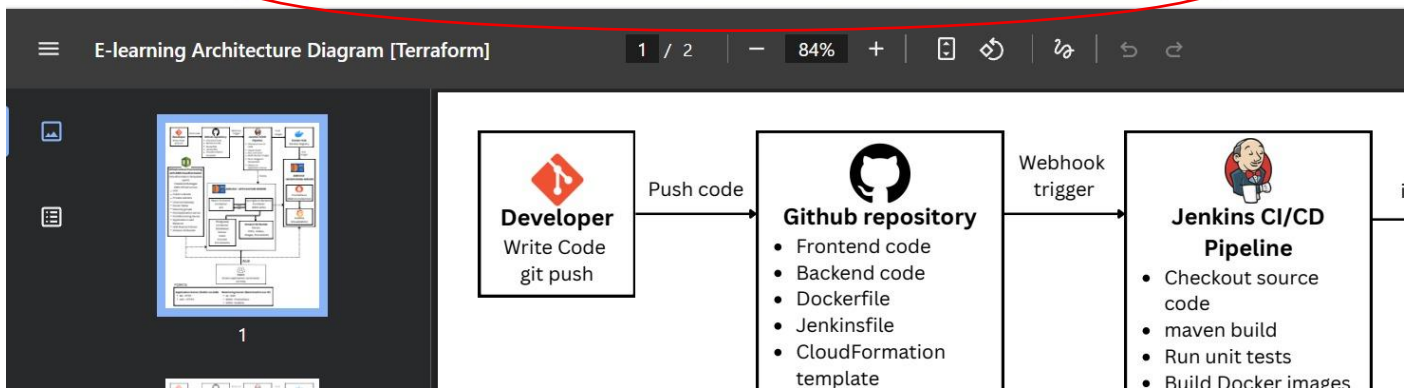
- Add HTTPS using a custom domain and SSL/TLS certificates.
- Introduce Application Load Balancer and Auto Scaling Groups.
- Move PostgreSQL to Amazon RDS for managed database operations.
- Add CloudFront for S3 content delivery optimization.
- Implement centralized logging using CloudWatch or ELK.
- Add Trivy image scanning and SonarQube quality gates.
- Move deployment to Kubernetes or Amazon EKS.
- Add application features such as quizzes, progress tracking and certificates.

APPENDIX

A.1 Screenshots







aws | Search [Alt+S] | Asia Pacific (Mumbai) | Santhoshi (697568497549) | AdminUser

Amazon S3 > Buckets > elearning-storage-coursecontentbucket-htmfhqkv9sh

Amazon S3

- Buckets**
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- Files**
 - File systems
- Access management and security**
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer

elearning-storage-coursecontentbucket-htmfhqkv9sh

Objects | Metadata | Properties | Permissions | Metrics | Management | File systems - new | Access Points

Objects (1)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix: Show versions: ☐

☐	Name	Type	Last modified	Size	Storage class
☐	course-1/	Folder	-	-	-

aws | Search [Alt+S] | Asia Pacific (Mumbai) | Santhoshi (697568497549) | AdminUser

EC2 > Instances

EC2

- Dashboard
- AWS Global View
- Events
- Instances**
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager
- Images**
 - AMIs
 - AMI Catalog
- Elastic Block Store**
 - Volumes
 - Snapshots
 - Lifecycle Manager
- Network & Security**
 - Security Groups

Instances (7)

Last updated less than a minute ago | Connect | Instance state | Actions | Launch instances

Saved filter sets: Choose filter set | Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☐	elearning-backend	i-0f4bea138e7e3ea3d	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-52-6
☐	elearning-frontend	i-0f8308b12d7cd4b3b	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-15-2
☐	elearning-jenkins	i-0aa985fdd3b18e82f	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-15-2
☐	elearning-monitoring	i-00f704a165fa198f1	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-52-6

All instances

Instance alarm states
 In alarm
 None Show instances in alarm

Service health
 EC2 service status
 Operating normally

Health events affecting EC2 instances
 Open events
 None Show instances
 Upcoming events
 None Show instances

View in CloudWatch | View Service Health Dashboard | View AWS Health Dashboard